Exercise 12

Elisa Schöpf, Leon Krieger, Nikola Adzic

June 10, 2026

Contents

1 Setup and Basic Execution **1**

2 Profiling **1**

3 Code Understanding **2**

4 Optimization **2**

1 Setup and Basic Execution

LCC3 fib.lua results:

```
100 x fibonacci_naive(30)     time:  13.7866 s  --  832040
10000000 x fibonacci_tail(30) time:  15.8393 s  --  832040
25000000 x fibonacci_iter(30) time:  12.2847 s  --  832040
```

The median of averages from 20 runs was 13.7609 s

2 Profiling

Initially, only the benchmark was profiled using profile.lua (`https://github.com/2dengine/profile.lua`). This however only shows a profile of the functions, making it unusable for optimizations on the lua interpreter itself.
After that, `perf` was used. The results were more informative:

```
[...]
# Samples: 116K of event 'cycles:u'
# Event count (approx.): 114068069768
#
# Children      Self  Command  Shared Object     Symbol
# ........  ........  .......  .................  ......................
#
    79.12%    78.18%  lua      lua               [.] luaV_execute
            |
            |--14.88%--0
            |          |
            |           --14.61%--luaV_execute
            |
            |--7.37%--0x281c900060080
            |          luaV_execute
[...]
```

Additionally, manual counting of OPs was included to see, which OPs were called the most:

```
--- Opcode Statistics ---
EQI             : 310000031
LTI             : 271946237
GETTABUP        : 135973068
ADD             : 135973068
MOVE            : 35000103
LOADI           : 25000001
CLOSURE         : 10000001
VARARGPREP      : 4
SETTABUP        : 3
--------------------------
```

3 Code Understanding

As seen on the `perf` report, the interpreter spends a lot of time in `luaV_execute`. Inside that is a large switch statement which checks against different OP_IDs to determine what to do next. As expected, this takes a lot of time and is inefficient.
`LUA_USE_JUMPTABLE` helps with that by enabling direct-threaded dispatch in the virtual machine using GCC/Clang computed gotos. This replaces the large switch statement, thus increasing performance.

4 Optimization

Applying heuristics to OP_ADD resulted in 1.4% performance increase Using all optimizations above and -fprofile-generate + -fprofile-use resulted in 9.6% performance increase